MinuteAI — Executive Meeting Intelligence

End-to-End Automated Audio-to-Minutes Synthesis, Speaker Diarization & Analytics

Academic Defense Level: B.Tech 3rd Year Minor Project	**Stack:** Python 3.12, Whisper, pyannote, FastAPI	**Verification:** 21/21 Unit & Integration Tests Passing

Project Abstract & Executive Overview

MinuteAI is a production-grade meeting intelligence system engineered to eliminate the manual overhead of meeting note-taking. The platform ingests conversational multi-speaker audio recordings in any standard format (WAV, MP3, MPEG, M4A, FLAC, OGG) and autonomously synthesizes executive-ready meeting minutes. Unlike commercial cloud solutions (such as Otter.ai or Fireflies.ai) that transmit corporate discussions to third-party servers, MinuteAI executes **100% locally and securely** with complete data privacy, providing speaker-attributed transcripts, topic segmentation, consensus decisions, structured action-item matrices (with owners, deadlines, and priorities), talk-time participation metrics, and multi-format document exports (PDF, Word DOCX, Markdown, Text, JSON).

1. Team Members & Technical Deliverables

The project was designed with a decoupled 4-module architecture, enabling rigorous individual accountability and seamless integration:

Team Member	Assigned Domain	Core Implementation Files	Viva Defense Focus
Member 1 **Annyasha Bhul**	Audio Preprocessing & Speech-to-Text	src/transcription/audio_preprocessing.py src/transcription/whisper_transcription.py src/transcription/evaluate_wer.py	16 kHz Mono conversion, -20 dBFS volume normalization, Whisper Transformer encoder-decoder, CUDA GPU vs CPU execution, WER metric.
Member 2 **Subhadip Santra**	Speaker Diarization & Integration	src/diarization/speaker_diarization.py src/diarization/integrate_transcription.py src/diarization/evaluate_der.py	pyannote.audio voice embeddings, offline spectral clustering fallback, temporal interval overlap alignment, DER metric & Hungarian bijection.
Member 3 **Sonali Jana**	NLP & Information Extraction	src/summarization/meeting_summarizer.py Thematic topic segmentation Decision & action item rules	Conversational filler word removal, consensus decision detection, rule-based action items (assignees, deadlines, priorities), ROUGE/BLEU scores.
Member 4 **Shreyasi Mondal**	Backend, Frontend & Exporters	src/backend/app.py src/frontend/ (HTML, CSS, JS) src/utils/document_exporter.py	FastAPI asynchronous REST endpoints, Glassmorphism UI, clickable audio seeking synchronization, ReportLab PDF and python-docx generation.

2. System Architecture & 5-Stage Processing Pipeline

MinuteAI processes incoming meeting recordings through 5 decoupled, fault-tolerant processing stages:

Pipeline Visual Dataflow

Architectural Dataflow Pipeline:

[Raw Audio File] (WAV / MP3 / MPEG / M4A / FLAC / OGG)
↓
[Stage 1: Preprocessing] → *16 kHz Mono PCM WAV + -20 dBFS Volume Normalization*
↓ (Forked dual-stream acoustic processing)
• **[Stage 2: Speech-to-Text]** → *OpenAI Whisper produces timestamped word utterances*
• **[Stage 3: Speaker Diarization]** → *pyannote.audio / Spectral Clustering segments speaker turns*
↓
[Stage 4: Interval Overlap Integration] → *Assigns text to speakers via Maximum Temporal Overlap*
↓
[Stage 5: NLP Summarization & Document Export] → *Summary, Decisions, Action Items → PDF / DOCX / Web UI*

Pipeline Stage	Technical Mechanism & Libraries	Input / Output & Functionality
Stage 1: Preprocessing	pydub, FFmpeg binary auto-discovery, Nyquist-Shannon 16 kHz downsampling	Converts WAV, MP3, MPEG, M4A to uncompressed 16-bit Mono PCM WAV. Applies gain normalization to -20 dBFS to balance participant vocal energy.
Stage 2: Speech-to-Text	OpenAI Whisper (base/tiny/small), PyTorch, Mel-spectrogram front-end	Generates timestamped sentence tokens [start_time, end_time]. Automatically dispatches to CUDA GPU or multi-threaded CPU with fp16=False stability.
Stage 3: Diarization	pyannote.audio + Scikit-Learn Acoustic Spectral Clustering fallback	Extracts speaker turns [t_start, t_end, speaker_id]. If Hugging Face token is missing or network is offline, activates local acoustic feature clustering.
Stage 4: Interval Overlap	Temporal Overlap Intersection Algorithm: Overlap = max(0, min(e_w, e_d) - max(s_w, s_d))	Maps each Whisper utterance to the speaker possessing maximum temporal overlap. Resolves ambiguous pauses and overlapping speech boundaries cleanly.
Stage 5: NLP & Export	Regex NLP, reportlab, python-docx, FastAPI REST API	Filters filler disfluencies ('um', 'uh', 'you know'), clusters topics, extracts consensus decisions, creates action matrix, exports to PDF, DOCX, TXT, JSON.

3. Mathematical Evaluation Metrics & Empirical Benchmarks

MinuteAI incorporates rigorous academic benchmarks to objectively evaluate Speech-to-Text accuracy, diarization fidelity, and summarization content retention against human-verified ground-truth transcripts:

Evaluation Metric Formulations

1. **Word Error Rate (WER) & Word Accuracy:**
   WER = (Substitutions + Deletions + Insertions) / Total Reference Words
   Word Accuracy = 1 - WER = Hits / Total Reference Words

2. **Diarization Error Rate (DER):**
   DER = (Missed Speech Time + False Alarm Time + Speaker Confusion Time) / Total Vocal Time
   **Optimal cluster-to-ground-truth bijection is resolved using the Hungarian Maximum Weight Matching algorithm.*

3. **ROUGE & BLEU Scores:**
   ROUGE-N: N-gram recall overlap between generated summary and human minutes reference.
   ROUGE-L: Longest Common Subsequence (LCS) F1 score capturing sentence structure coherence.
   BLEU: Geometric mean of modified n-gram precisions penalized by length Brevity Penalty (BP).

Empirical Performance Benchmark on Reference Meeting Datasets:

Evaluation Dimension	Standard Metric	Meeting 01 (Sprint)	Meeting 02 (Website)	Combined Mean
Speech-to-Text (ASR)	Word Error Rate (WER)	12.90%	3.80%	**8.35%**
Speech-to-Text (ASR)	Word Accuracy (1 - WER)	87.10%	96.20%	**91.65%**
Speech-to-Text (ASR)	Character Error Rate (CER)	7.84%	1.17%	**4.51%**
Speaker Diarization	Turn Attribution Accuracy	65.00%	44.44%	**54.72%**
Speaker Diarization	Time-Weighted Accuracy	65.05%	38.31%	**51.68%**
Summarization & NLP	ROUGE-1 F1 Score	63.67%	63.84%	**63.76%**
Summarization & NLP	ROUGE-2 F1 Score	45.49%	50.13%	**47.81%**
Summarization & NLP	ROUGE-L F1 Score	53.88%	57.36%	**55.62%**
Summarization & NLP	BLEU Precision Score	30.33%	43.06%	**36.70%**

4. Project Repository & Directory Structure

The project follows professional modular software engineering standards, completely separating concerns across audio, AI, backend, frontend, and test suites:

Subdirectory / Module	Key Files	Functionality & Architectural Role
src/transcription/	audio_preprocessing.py whisper_transcription.py evaluate_wer.py	Ingests multi-format audio, downsamples to 16 kHz Mono PCM, balances gain to -20 dBFS, executes Whisper ASR with CUDA/CPU dispatch, and calculates Word Error Rate.
src/diarization/	speaker_diarization.py integrate_transcription.py evaluate_der.py	Extracts speaker voice turns using pyannote.audio or local spectral acoustic clustering, calculates temporal overlap with Whisper words, and scores DER.
src/summarization/	meeting_summarizer.py	Filters filler disfluencies, segments discussions by semantic topics, extracts explicit decisions, and synthesizes action items with assignees, deadlines, and priorities.
src/backend/	app.py	FastAPI asynchronous server managing pipeline execution, static UI serving, audio streaming, dynamic speaker renaming, and multi-format file downloads.
src/frontend/	index.html, style.css app.js	Responsive Glassmorphism web dashboard featuring interactive audio player with timestamp seek, talk-time analytics, speaker renamer, and multi-format export buttons.
src/utils/	document_exporter.py	Generates formatted corporate PDF reports (ReportLab) and editable Microsoft Word documents (python-docx), Markdown, JSON, and Plain Text files.
tests/	test_pipeline.py test_diarization.py	Comprehensive unit and integration test suites validating end-to-end functionality (21/21 tests passing).

Technology Stack Specifications

- **Programming Language:** Python 3.12+ (64-bit)
- **Core AI Engines:** OpenAI Whisper (ASR), pyannote.audio & Scikit-Learn (Diarization)
- **Audio Signal Processing:** pydub, FFmpeg, soundfile, scipy
- **Web API & Server:** FastAPI, Uvicorn, ASGI Asynchronous Event Loop
- **Document Synthesis:** ReportLab 5.1 (PDF Engine), python-docx (Microsoft Word)
- **Evaluation Engine:** jiwer (WER/CER), pyannote RTTM (DER), rouge-score, NLTK (BLEU)

5. Execution Guide & REST API Reference

MinuteAI provides two distinct modes for running and demonstrating the project, making it ideal for both low-spec viva machines and high-impact seminar presentations:

Two Demonstration Modes

Mode 1: Terminal Demonstration (Recommended for Viva Voce & Low-Spec Laptops)

- Launch with: python main.py or double-click start_terminal_demo.bat
- Step-by-step console visualization of audio preprocessing, Whisper transcription, speaker diarization, and minutes synthesis.
- Automatically exports formatted PDF, Word (.docx), and TXT files directly to data/meeting_minutes/.

Mode 2: Modern Executive Web Dashboard (Recommended for PPT Presentation & Project Exhibition)

- Launch with: python run_app.py or double-click start_app.bat
- Automatically opens browser at: **http://127.0.0.1:8000**
- Features: 1-click sample demo buttons, audio player seek sync on timestamp click, interactive speaker renaming, and real-time talk-time charts.

FastAPI REST Endpoints Reference:

HTTP Method	Endpoint URL	Description & Response
GET	/health	Performs system diagnostics, detects CPU/CUDA hardware, checks FFmpeg discovery.
GET	/api/samples	Lists pre-loaded benchmark sample audio files with duration and speaker counts.
POST	/api/process-meeting	Accepts multipart audio file upload (WAV, MP3, MPEG, M4A) and executes full 5-stage pipeline.
POST	/api/process-sample/{id}	Executes or retrieves cached pipeline results for pre-loaded benchmark meetings.
GET	/api/meetings/{id}	Returns complete structured meeting object: minutes, diarized transcript, talk-time stats.
GET	/api/export/{id}/{format}	Generates and streams downloadable document: pdf, docx, txt, md, json.
POST	/api/rename-speakers/{id}	Dynamically updates speaker names across transcripts and re-exports documents instantly.
GET	/api/evaluate/{id}	Computes real-time WER, DER, ROUGE-1/2/L, and BLEU against human ground-truth transcript.

6. Core Algorithm Cheatsheet (For Viva Whiteboard)

Examiners frequently ask candidates to write core algorithmic logic on a blackboard or sheet of paper to verify hands-on programming understanding. Here are the two most critical algorithms implemented in MinuteAI:

Snippet A: Audio Ingestion, Resampling & Normalization (pydub)

```
from pydub import AudioSegment

def preprocess_audio(input_file: str, output_wav: str) -> str:
    # 1. Ingest audio in any format (WAV, MP3, MPEG, M4A, FLAC)
    audio = AudioSegment.from_file(input_file)
    # 2. Resample to 16 kHz Mono PCM (matches Whisper front-end)
    audio = audio.set_channels(1).set_frame_rate(16000)
    # 3. Dynamic peak/RMS gain normalization to -20 dBFS
    gain_change = -20.0 - audio.dBFS
    normalized = audio.apply_gain(gain_change)
    # 4. Export standard 16-bit PCM WAV
    normalized.export(output_wav, format="wav")
    return output_wav
```

Snippet B: Interval Overlap Integration (Aligning Whisper Words to Diarization Turns)

```
def align_words_to_speakers(whisper_segments, diarization_turns):
    aligned_transcript = []
    for seg in whisper_segments:
        s_start, s_end, text = seg["start"], seg["end"], seg["text"]
        best_speaker, max_overlap = "SPEAKER_00", 0.0
        for turn in diarization_turns:
            # Overlap = max(0, min(s_end, turn.end) - max(s_start, turn.start))
            overlap = max(0.0, min(s_end, turn["end"]) - max(s_start, turn["start"]))
            if overlap > max_overlap:
                max_overlap = overlap
                best_speaker = turn["speaker"]
        aligned_transcript.append({
            "speaker": best_speaker,
            "start": s_start,
            "end": s_end,
            "text": text
        })
    return aligned_transcript
```

Whiteboard Tip for Viva Defense

When explaining Snippet B on the board, draw two horizontal time bars: one for the Whisper sentence time window and one for the Speaker Diarization turn. Highlight the intersection interval and explain how computing min(end_w, end_d) - max(start_w, start_d) accurately resolves turn ownership even when sentence boundaries slightly precede or lag the speaker's vocal acoustic turn.

7. Viva Voce Defense Guide — Part 1: Audio, STT & Diarization

Top examination questions and high-scoring model answers for project reviews and external viva examiners:

Q1: Why do we convert input audio to 16 kHz Mono PCM WAV?

Answer: Human speech frequencies and vocal formants predominantly lie below 8 kHz. Under the **Nyquist-Shannon Sampling Theorem**, a signal must be sampled at at least twice its highest frequency (2 x 8 kHz = 16 kHz). Higher rates (such as 44.1 kHz used in music) unnecessarily quadruple memory consumption without yielding higher speech recognition accuracy. Converting stereo to mono (1 channel) eliminates channel phase cancellation and simplifies acoustic feature extraction.

Q2: What is -20 dBFS volume normalization and why is it essential?

Answer: dBFS represents *Decibels relative to Full Scale*, where 0 dBFS is the maximum digital amplitude before clipping distortion. Normalizing conversational audio to -20 dBFS balances volume between quiet and loud speakers, ensuring that Whisper's 80-channel log-Mel spectrogram receives consistent acoustic energy, which significantly reduces deletion errors on faint utterances.

Q3: How does OpenAI Whisper transcribe speech accurately?

Answer: Whisper is an **Encoder-Decoder Transformer** trained on 680,000 hours of multilingual weakly-supervised audio. The audio is segmented into 30-second windows and converted into an 80-channel log-Mel spectrogram. The Transformer Encoder extracts acoustic representations using 1D convolutions and self-attention, and the Autoregressive Transformer Decoder predicts text tokens with cross-attention alongside special tokens for language, timestamp boundaries, and text transcription.

Q4: What is Speaker Diarization and how does pyannote.audio perform it?

Answer: Diarization answers *'Who spoke when'*. `pyannote.audio` utilizes a pre-trained convolutional/recurrent neural network to extract continuous voice embeddings (x-vectors / d-vectors) across a sliding temporal window. It then computes pairwise cosine distances and clusters the embeddings into distinct speaker identities.

Q5: What happens if there is no internet connection or Hugging Face token?

Answer: Unlike fragile systems that crash offline, MinuteAI incorporates an **Acoustic Spectral Clustering Fallback**. It extracts acoustic audio features (spectral centroid, zero-crossing rate, energy variance, spectral rolloff) across 1-second frames and partitions speaker turns using Scikit-Learn's Agglomerative Clustering, ensuring 100% offline uptime.

Q6: How does the system align Whisper utterances with speaker turns?

Answer: We engineered the **Maximum Temporal Interval Overlap Algorithm**. For each Whisper utterance [t_start, t_end], we compute its intersection with every diarization turn [d_start, d_end] using: Overlap = max(0, min(t_end, d_end) - max(t_start, d_start)). The utterance is assigned to the speaker exhibiting the largest positive temporal overlap.

7. Viva Voce Defense Guide — Part 2: NLP, Metrics & System Defense

Advanced technical questions covering NLP extraction, mathematical evaluation, privacy advantages, and UI architecture:

Q7: How does MinuteAI extract structured action items and assignees?

Answer: The NLP module employs rule-based pattern matching. It identifies modal commitment verbs ('will', 'shall', 'agreed to', 'needs to', 'action item'), extracts the task predicate, associates it with the active speaker or explicitly named attendees, and scans for temporal markers ('by Friday', 'tomorrow', 'next week') to populate the task matrix with owner, deadline, and priority.

Q8: What is Word Error Rate (WER) and how is it calculated?

Answer: WER is the industry standard for ASR evaluation: WER = (S + D + I) / N, where S is substitutions, D is deletions, I is insertions, and N is total words in the reference transcript. Word Accuracy is 1 - WER. MinuteAI achieves an average WER of **8.35%** (91.65% Word Accuracy).

Q9: What is the difference between ROUGE and BLEU metrics?

Answer: ROUGE is recall-oriented and measures what proportion of reference n-grams appear in the generated summary (evaluating completeness). **BLEU** is precision-oriented and measures what proportion of generated n-grams are correct against reference text (penalizing hallucination with a Brevity Penalty). Both are essential to confirm summary fidelity.

Q10: Why build an on-premise local pipeline instead of using Otter.ai or Fireflies.ai?

Answer: Commercial tools upload confidential board meetings and IP discussions to foreign cloud servers, creating severe compliance risks (GDPR, HIPAA, corporate confidentiality). MinuteAI runs **100% locally on-device** with zero data leakage, zero monthly API subscriptions, and full autonomy.

Q11: How does the web audio player sync when clicking a transcript timestamp?

Answer: Each transcript row in the HTML DOM stores data attributes (e.g. data-start='15.4'). When clicked, an event listener sets the HTML5 audio element's audio.currentTime = start_seconds and calls audio.play(), providing instant seek synchronization.

Q12: How are unit and integration tests verified?

Answer: The repository includes 21 comprehensive automated tests in tests/test_pipeline.py and tests/test_diarization.py covering audio preprocessing, Whisper transcription, diarization fallbacks, interval overlap edge cases, filler filtering, and PDF generation. All 21 tests pass with 100% green status.

Final Viva Defense Summary Checklist

✓ **Running the Demo:** Show start_terminal_demo.bat or start_app.bat in action.
✓ **Highlighting Key Strengths:** Emphasize 100% on-device privacy, 8.35% WER accuracy, and the offline diarization fallback.
✓ **Document Exports:** Demonstrate that MinuteAI immediately produces formatted PDF, Word (.docx), and text deliverables.